



Prueba de desempeño ASP.NET C#

El siguiente planteamiento detalla la funcionalidad mínima viable (MVP) solicitada por el cliente. Si bien lograr que el sistema cumpla con estos requerimientos es indispensable, el alcance de esta prueba es intencionalmente corto para darte libertad de acción.

El verdadero diferenciador en tu entrega no será únicamente "hacer que funcione", sino tu criterio como desarrollador y el valor adicional que aportes a la solución.

Queremos evaluar tu visión de producto y tu madurez técnica. El mayor peso de nuestra evaluación recaerá en aquellas iniciativas, decisiones arquitectónicas o mejoras al flujo que decidas incorporar de manera proactiva, porque consideras que son vitales para un ecosistema de software real, robusto y escalable.

Planteamiento del Problema

En el ecosistema actual de rentas cortas, existe una brecha significativa entre la experiencia de búsqueda fluida que demandan los huéspedes y las garantías de seguridad y rentabilidad que exigen los propietarios.

Por un lado, los procesos tradicionales de reserva carecen de validaciones de identidad robustas en tiempo real, lo que incrementa el riesgo de fraude y daños a la propiedad. Por otro lado, los dueños de los inmuebles a menudo gestionan sus propiedades "a ciegas", careciendo de herramientas centralizadas que les ofrezcan métricas claras sobre el rendimiento financiero de sus activos o facilidades para la conciliación contable. Se requiere una plataforma unificada que elimine la fricción en el proceso de reserva, automatice la validación de identidad (KYC) mediante Inteligencia Artificial, permita a los usuarios curar sus intereses a su propio ritmo, y empodere a los propietarios con análisis de datos precisos para la toma de decisiones.

Requerimientos Generales del Sistema

- **Gestión de Disponibilidad Estricta:**
El sistema debe garantizar que no existan reservas solapadas para un mismo inmueble en las mismas fechas (prevención de *double-booking*).
- **Estandarización de Horarios:**
Toda reserva confirmada debe parametrizar automáticamente la hora de llegada (Check-in) a las 2:00 PM y la hora de salida (Check-out) a las 12:00 PM (mediodía).



- **Sistema Omnicanal de Alertas:** Implementación de un motor de notificaciones capaz de despachar alertas por correo electrónico y notificaciones dentro de la aplicación para eventos clave (confirmación de reserva, validación de identidad, recordatorios de llegada/salida entre otros). *aquí pueden usar laravel*
- **Privacidad y Seguridad de Datos:** Protección criptográfica y eliminación segura de los documentos de identidad suministrados durante el proceso de validación.

Requerimientos Específicos por Rol

1. Rol: Usuario (Arrendatario / Huésped)

- **Búsqueda y Exploración:** El usuario debe poder visualizar un catálogo de inmuebles y filtrar los resultados por ubicación geográfica y rango de fechas de disponibilidad sin necesidad de estar registrado.
- **Gestión de Favoritos (Wishlist):** El usuario debe poder marcar y desmarcar inmuebles de su interés para guardarlos en una lista personalizada. Esta lista debe ser de fácil acceso y permitir al usuario comparar o reservar rápidamente propiedades que haya preseleccionado en sesiones anteriores.
- **Autenticación Diferida:** El sistema debe permitir la navegación anónima y solicitar el inicio de sesión o registro únicamente cuando el usuario decida efectuar el pago, confirmar una reserva, o guardar un inmueble en su lista de favoritos de forma permanente.
- **Validación de Identidad Asistida por IA (KYC):**
 - El usuario debe poder capturar y subir fotografías de su documento de identidad (cédula) desde su dispositivo móvil.
 - El sistema debe procesar la imagen mediante Inteligencia Artificial para extraer de forma autónoma: Nombres, Apellidos, Número de Documento y Fecha de Nacimiento.
 - El sistema debe emitir un veredicto de validación (aprobado/rechazado) antes de permitir la finalización de la primera reserva.
- **Gestión de Estancia:** El usuario debe poder visualizar el estado de sus reservas, incluyendo las fechas y las políticas de horario estándar (14:00 - 12:00).



2. Rol: Dueño (Propietario / Anfitrión)

- **Gestión de Inventario:**
El dueño debe contar con un módulo dedicado para publicar nuevos inmuebles, editar características, subir material fotográfico y establecer tarifas.
- **Dashboard de Rendimiento:**
El sistema debe proveer un panel de control interactivo que consolide las métricas de los inmuebles. Debe incluir indicadores de rentabilidad, tasas de ocupación e ingresos generados en periodos de tiempo seleccionables.
- **Extracción de Datos:**
El dueño debe poder generar y descargar reportes en formato Excel (.xlsx).
 - El reporte debe poder generarse para la totalidad del portafolio o filtrarse por un inmueble en particular.
 - El documento exportado debe contener: fechas de alquiler, precio pagado, datos básicos del usuario que rentó y el inmueble asociado.

Resultados Esperados (Qué se espera lograr)

1. **Mitigación de Riesgos:**
Reducir a casi cero la suplantación de identidad o el fraude en las reservas mediante la barrera automatizada de la IA, brindando total tranquilidad a los dueños.
2. **Conversión Optimizada:**
Al posponer el inicio de sesión hasta el momento de la reserva, se espera mantener al usuario interesado (menor tasa de rebote) mientras explora el catálogo libremente.
3. **Fidelización y Retención:**
Al introducir la función de favoritos, se facilita que los usuarios guarden opciones a su propio ritmo, fomentando su regreso a la plataforma y aumentando la probabilidad de que una simple exploración se convierta en una reserva a mediano plazo.
4. **Transparencia Financiera:**
Entregar a los propietarios una visión clara y en tiempo real de si su inversión está siendo rentable, facilitando además su contabilidad mensual a través de las exportaciones en Excel.
5. **Operación Autónoma:**
Lograr que la interacción entre dueños y arrendatarios fluya sin necesidad de intervención de soporte humano, delegando la carga operativa al



sistema de disponibilidad, horarios fijos y notificaciones automáticas.

Requerimientos de Desarrollo

- **Core Obligatorio:** El núcleo y la lógica principal del sistema deben desarrollarse en **.NET 9 o .NET 10**.
- **Tecnologías Complementarias:** El uso de **Laravel o Node.js** es válido, pero estrictamente como un pequeño complemento para tareas secundarias (ej. microservicios ligeros).
- **Infraestructura:** Es obligatorio usar **Docker o Docker Compose** para poder levantar todo el entorno de la aplicación de forma sencilla.

Entregables

1. **Código Fuente:** Entregar mediante un enlace a un **repositorio Git** y en un **archivo comprimido** (asegúrate de excluir carpetas de dependencias o compilados como bin, obj, node_modules, vendor).
2. **Archivo README.md:** Debe incluir en la raíz del proyecto:
 - a. Requisitos previos.
 - b. Comandos exactos para levantar el proyecto con Docker.
 - c. Una breve explicación de la arquitectura y cómo se abordaron los problemas técnicos de la prueba.